This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TIE.2014.2365752, IEEE Transactions on Industrial Electronics

A Network Tearing Technique for FPGA-Based Real-Time Simulation of Power Converters

Tarek Ould-Bachir, *Member, IEEE*, Handy Fortin Blanchette, *Member, IEEE*, and Kamal Al-Haddad, *Fellow Member, IEEE*

***Abstract*—The realm of Hardware-in-the-Loop (HIL) simulation resorts to Field Programmable Gate Arrays (FPGAs) to achieve time-steps below 1 μs. Such low time-steps are of importance for the aerospace and automotive industries, where power converters have their switching frequencies in the 10-200 kHz range. This article proposes a Network Tearing Technique (NTT) that allows subsets of switches to be treated independently, alleviates embedded memory requirements, and reduces the computational burden. An iterative algorithm is used to determine the state of naturally commutated switches, thus offering a realistic model of the power converter, independently of its operation mode or topology. A Gauss-Jordan processing unit is implemented to solve interface voltages/currents from the torn circuit. Custom floating-point operators are used to ensure good accuracy, high frequency operation as well as low computational latency. A neutral-point-clamped (NPC) converter case study is presented to demonstrate the effectiveness of the method. Simulation results are validated against a reference model at a 750 ns time-step and 30 kHz Sine Pulse Width Modulation (SPWM) switching frequency.**

***Index Terms*—Real-time simulation, network tearing technique, FPGA, floating-point arithmetic.**

I. Introduction

HARDWARE-IN-THE-LOOP simulation is a prototyping approach that aims to reduce the design cost of power systems by running comprehensive tests early in the development stage [1]. However, modelling a power converter is a challenging task because of the varying topology of the circuit. Moreover, modern technologies seek to increase the power density of the converter and to reduce the total harmonic distortion (THD) by targeting switching frequencies in the $10-200$ kHz range [2]–[4]. Such frequencies require time-steps below 1 μs, which are hardly achievable in the context of CPU-based simulation, where time-steps are in the $5-10$ μs range at best [5]. FPGA-based simulation solves this issue by allowing very low time-steps to be reached [6]–[8].

The simulation of switching networks resorts to one of the following switch models [9]: 1) the ideal model; 2) the switching function model; or 3) the average model. When detailed transients at switching events are of interest, the ideal switch model is the preferred approach. However, factorization is hardly conceivable in an FPGA-based solution, so the ideal switch model approach usually implies storing all network equations for every switch status combination [10]. Such an approach limits the number of switches to 6-8. In the early 1990s, a switch model that keeps the network equations fixed regardless of switch statuses was proposed [11], [12]. The fixed-matrix modelling approach has successfully been implemented on FPGA [6], [8], [13], but it is known to introduce false numerical transients due to its convergence process that restricts its operation at high switching frequencies [14].

This paper proposes a Network Tearing Technique that splits a given circuit into separate parts, thus allowing subsets of switches to be treated independently. Such an approach increases the potential number of switches present in the circuit, alleviates on-chip memory requirements, and reduces the computational burden. Neither of these results is obtained by compromising the accuracy of the simulation. The proposed method is comparable in its principles to [15]–[19]; the originality of the presented research resides in the fact that it is tailored for FPGA-based simulations. Contributions of the paper also comprise an original FPGA-based computing engine composed of multiple processing units (PUs), among which a new and highly pipelined Gauss-Jordan processor that is responsible for solving interface variables of the torn circuit. All PUs use custom floating-point operators to ensure computational accuracy and a large dynamic range.

The remainder of this paper is organized as follows. The NTT-based power converter modelling is presented in Section II. Forced and naturally commutated switches' logic update is discussed in Section III. The content of Sections II and III are used in Section IV to model a Neutral Point Clamped (NPC) converter. Section V presents the hardware architecture for solving the NPC equations in real-time, reports its area utilization and speed performance. Section V also provides and discusses FPGA-based simulation results. Section VI concludes this work.

II. Modelling Approach

A. Illustrative Example

The single-phase inverter connected to an inductive load shown in Fig. 1.a is used as an example to introduce the proposed network tearing method, which proceeds as follows. The companion-circuit-based nodal analysis [20] is used for obtaining network equations. The circuit is then split into multiple sub-circuits to reduce the computational burden and memory requirements. Each sub-circuit is therefore reduced

Manuscript received March 12, 2014; revised July 14, 2014 and August 26, 2014; accepted October 4, 2014.

Tarek Ould-Bachir, Handy Fortin Blanchette, and Kamal Al-Haddad are with the Departement of Electrical Engineering, École de Technologie Supérieure, Montreal, Canada (e-mail: ouldbachir@ieee.org, Handy.Fortin-Blanchette@etsmtl.ca, Kamal.Al-Haddad@etsmtl.ca).

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TIE.2014.2365752, IEEE Transactions on Industrial Electronics

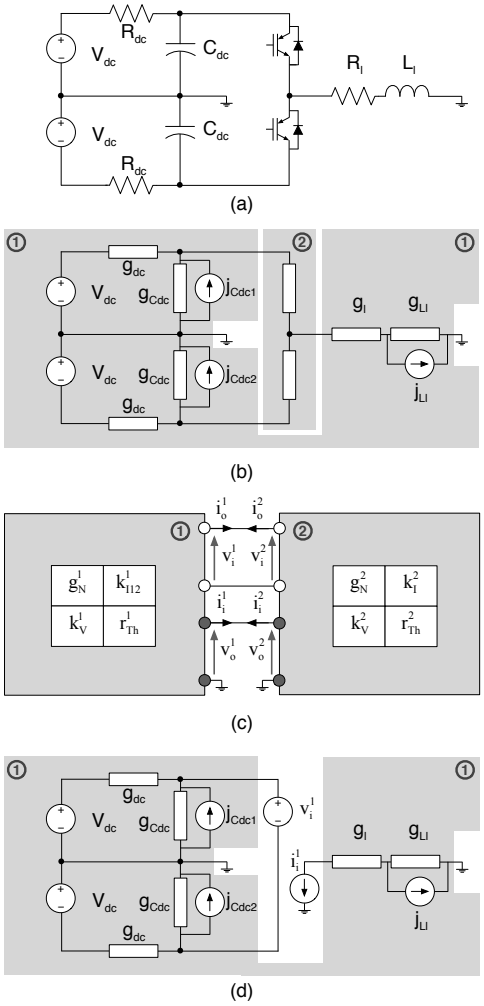

Fig. 1. Single phase inverter example: (a) Original circuit; (b) Original circuit with components replaced by their companion circuits, partitioning is highlighted; (c) Two two-port hybrid equivalents connected to solve interface voltage/currents; (d) sub-circuit (1) connected to voltage and current sources.

to an equivalent Norton/Thévenin circuit and interface currents/voltages connecting the equivalents are evaluated. Each sub-circuit is then connected to voltage and current sources with values from the previous step to solve for nodal voltages, branch currents as well as the various state variables. This final step can be performed in parallel.

The procedure is illustrated in Fig. 1. Fig. 1.a presents the original circuit. Fig. 1.b presents the same circuit where components of the circuit are replaced by their companion circuit: capacitors and inductors are replaced by a conductance with a parallel current source known as the history term, responsible for modelling the energy stored in the component. Switches are replaced by equivalent conductances ($1/R_{on}$ or $1/R_{off}$, depending on switch statuses). Fig. 1.b also shows the proposed partitioning of the converter into two separate sub-circuits. Fig. 1.c shows the two two-port Norton/Thévenin equivalents used to solve for interface voltage ($\mathbf{v_i^1}$) and current ($\mathbf{i_i^1}$). Finally, Fig. 1.d illustrates the last step that consists in connecting each sub-circuit (only sub-circuit (1) is shown) with voltage and current sources, whose values were obtained from the previous step. The current and voltage sources replicate the contribution of the missing parts from the circuit.

The reduction of sub-circuits (1) and (2) to their hybrid Norton/Thévenin equivalents, as shown in Fig. 1.c, means that sub-circuit (1) is reduced to:

$$\begin{bmatrix} \mathrm{i}_\mathrm{o}^1 \\ \mathrm{v}_\mathrm{o}^1 \end{bmatrix} = \begin{bmatrix} \mathrm{i}_\mathrm{eq}^1 \\ \mathrm{v}_\mathrm{eq}^1 \end{bmatrix} - \begin{bmatrix} \mathrm{g}_\mathrm{N}^1 & \mathrm{k}_\mathrm{I}^1 \\ \mathrm{k}_\mathrm{V}^1 & \mathrm{r}_\mathrm{T}^1 \end{bmatrix} \begin{bmatrix} \mathrm{v}_\mathrm{i}^1 \\ \mathrm{i}_\mathrm{i}^1 \end{bmatrix}, \tag{1}$$

whereas sub-circuit (2) is reduced to:

$$\begin{bmatrix} \mathrm{i}_\mathrm{o}^2 \\ \mathrm{v}_\mathrm{o}^2 \end{bmatrix} = \begin{bmatrix} \mathrm{i}_\mathrm{eq}^2 \\ \mathrm{v}_\mathrm{eq}^2 \end{bmatrix} - \begin{bmatrix} \mathrm{g}_\mathrm{N}^2 & \mathrm{k}_\mathrm{I}^2 \\ \mathrm{k}_\mathrm{V}^2 & \mathrm{r}_\mathrm{T}^2 \end{bmatrix} \begin{bmatrix} \mathrm{v}_\mathrm{i}^2 \\ \mathrm{i}_\mathrm{i}^2 \end{bmatrix}. \tag{2}$$

Obviously, $[\mathrm{i}_\mathrm{eq}^2, \mathrm{v}_\mathrm{eq}^2] = [0, 0]$, since sub-circuit (2) is purely resistive. The hybrid nature of the equivalents is dictated by the fact that it is convenient to connect a given port to a voltage or a current source, knowing that the voltage or the current at that port is associated with the circuit's state variables. This allows relaxing the switch updating algorithm, to reduce the computational burden and the simulation time-step as well. These considerations are discussed further in this paper.

Each sub-circuit has at its ports two input variables (subscript i is used) and two output variables (subscript o is used), which yields a total of 8 unknowns. The network equations are built using the Tableau Approach [21]:

$$\mathbf{M}\mathbf{w} = \mathbf{z}, \tag{3}$$

where:

$$\mathbf{M} = \begin{bmatrix} +1 & 0 & 0 & 0 & \mathrm{g}_\mathrm{N}^1 & \mathrm{k}_\mathrm{I}^1 & 0 & 0 \\ 0 & +1 & 0 & 0 & \mathrm{k}_\mathrm{V}^1 & \mathrm{r}_\mathrm{Th}^1 & 0 & 0 \\ 0 & 0 & +1 & 0 & 0 & 0 & \mathrm{g}_\mathrm{N}^2 & \mathrm{k}_\mathrm{I}^2 \\ 0 & 0 & 0 & +1 & 0 & 0 & \mathrm{k}_\mathrm{V}^2 & \mathrm{r}_\mathrm{Th}^2 \\ 0 & 0 & 0 & 0 & -1 & 0 & +1 & 0 \\ 0 & 0 & 0 & 0 & 0 & +1 & 0 & +1 \\ -1 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & +1 & 0 & 0 & 0 & 0 \end{bmatrix} \tag{4}$$

and:

$$\begin{cases} \mathbf{w} &= \begin{bmatrix} \mathrm{i}_\mathrm{o}^1, & \mathrm{v}_\mathrm{o}^1, & \mathrm{i}_\mathrm{o}^2, & \mathrm{v}_\mathrm{o}^2, & \mathrm{v}_\mathrm{i}^1, & \mathrm{i}_\mathrm{i}^1, & \mathrm{v}_\mathrm{i}^2, & \mathrm{i}_\mathrm{i}^2 \end{bmatrix}^T \\ \mathbf{z} &= \begin{bmatrix} \mathrm{i}_\mathrm{eq}^1, & \mathrm{v}_\mathrm{eq}^1, & \mathrm{i}_\mathrm{eq}^2, & \mathrm{v}_\mathrm{eq}^2, & 0, & 0, & 0, & 0 \end{bmatrix}^T \end{cases} \tag{5}$$

The unknowns in Eq. 3 are then reduce to a set of independent variables, in this specific case $\mathbf{w}^{\mathrm{ind}} = [\mathrm{v}_\mathrm{i}^1, \mathrm{i}_\mathrm{i}^1]^T$. After permuting columns 5, 6 and 7, 8, a partial Gauss-Jordan reduction to the last two lines yields:

$$\begin{bmatrix} \mathrm{g}_\mathrm{N}^1 + \mathrm{g}_\mathrm{N}^2 & \mathrm{k}_\mathrm{I}^1 - \mathrm{k}_\mathrm{I}^2 \\ \mathrm{k}_\mathrm{V}^1 - \mathrm{k}_\mathrm{V}^2 & \mathrm{r}_\mathrm{Th}^1 + \mathrm{r}_\mathrm{Th}^2 \end{bmatrix} \begin{bmatrix} \mathrm{v}_\mathrm{i}^1 \\ \mathrm{i}_\mathrm{i}^1 \end{bmatrix} = \begin{bmatrix} \mathrm{i}_\mathrm{eq}^1 + \mathrm{i}_\mathrm{eq}^2 \\ \mathrm{v}_\mathrm{eq}^1 - \mathrm{v}_\mathrm{eq}^2 \end{bmatrix}. \tag{6}$$

One of the main contributions of this paper is the fact that Eq. 6 is a reduction of Eq. 4 in which all the entries are additions of terms from Eq. 1 and Eq. 2. This property allows for on-the-fly matrix assembly, which is an important asset when dealing with such problems on FPGA, more so when the computational time budget is as limited as in real-time simulation problems.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TIE.2014.2365752, IEEE Transactions on Industrial Electronics

B. FPGA-based Approach for Solving the MANA Equations

Detailed network equations are obtained using the Modified-Augmented Nodal Analysis (MANA) [20]. As do other companion-based circuit analysis methods, MANA proceeds by assembling network equations after discretizing all circuit devices using a numerical integration rule — the trapezoidal and the Backward Euler (BE) rules are typically used. The BE rule is preferable for the simulation of power electronic circuits because it avoids the numerical oscillations caused by the trapezoidal rule under switching conditions [12], [22]. With all circuit components replaced by their companion circuit, the nodal system of equations are assembled and given by [20]:

$$\mathbf{A}\mathbf{x} = \mathbf{b}, \tag{7}$$

where $\mathbf{A}$ is the augmented nodal matrix, $\mathbf{x}$ is a vector of unknown nodal voltages and branch currents, and $\mathbf{b}$ holds known voltage and current sources (including history terms).

To solve MANA equations on FPGA, Eq. 7 is reformulated in a state-space alike form [8]: let us denote $\mathbf{j}_n$ the vector containing the history terms in $\mathbf{b}_n$, and $\mathbf{u}_n$ the vector containing the remaining independent current/voltage sources; let $\mathbf{y}_n$ be the output vector containing branch currents and the voltages of interest; and let $\mathbf{A}^\sigma$ the matrix for a given combination σ of switch statuses. With $\mathbf{b}_n = \mathbf{K}_c[\mathbf{u}_n\ \mathbf{j}_n]^T$, where $\mathbf{K_c}$ is a connectivity matrix, one finds:

$$\mathbf{x}_n = (\mathbf{A}^\sigma)^{-1}\mathbf{K_c}\begin{bmatrix}\mathbf{u}_n\\ \mathbf{j}_n\end{bmatrix}, \tag{8}$$

and may obtain the following:

$$\begin{bmatrix}\mathbf{j}_{n+1}\\ \mathbf{y}_n\end{bmatrix} = \begin{bmatrix}\mathbf{W}^\sigma_{\mathbf{ju}} & \mathbf{W}^\sigma_{\mathbf{jj}}\\ \mathbf{W}^\sigma_{\mathbf{yu}} & \mathbf{W}^\sigma_{\mathbf{yj}}\end{bmatrix}\begin{bmatrix}\mathbf{u}_n\\ \mathbf{j}_n\end{bmatrix}, \tag{9}$$

where $\mathbf{W}^\sigma_{\mathbf{ju}}$, $\mathbf{W}^\sigma_{\mathbf{jj}}$, $\mathbf{W}^\sigma_{\mathbf{yu}}$, and $\mathbf{W}^\sigma_{\mathbf{yj}}$ are inline formulations derived from Eq. 8 [8].

C. The Multi-Port Hybrid Equivalent

The MPHE is a general formulation of the Norton/Thévenin equivalents. The hybrid nature of the MPHE results from the fact that the equivalent circuit encompasses voltage ports (Norton part) as well as current ports (Thévenin part). The circuit is supposed to contain resistances, current and voltage sources only — which is a reasonable assumption when discrete companion analysis (MANA) is used. For an $\mathcal{N}$-port circuit with $\mathcal{N}_\mathrm{V}$ voltage ports and $\mathcal{N}_\mathrm{I}$ current ports ($\mathcal{N}_\mathrm{V} + \mathcal{N}_\mathrm{I} = \mathcal{N}$), the MPHE of the circuit is given by:

$$\begin{bmatrix}\mathbf{i}_\mathrm{o}\\ \mathbf{v}_\mathrm{o}\end{bmatrix} = \begin{bmatrix}\mathbf{i}_\mathrm{eq}\\ \mathbf{v}_\mathrm{eq}\end{bmatrix} - \begin{bmatrix}\mathbf{G}_\mathrm{N} & \mathbf{K}_\mathrm{I}\\ \mathbf{K}_\mathrm{V} & \mathbf{R}_\mathrm{Th}\end{bmatrix}\begin{bmatrix}\mathbf{v}_\mathrm{i}\\ \mathbf{i}_\mathrm{i}\end{bmatrix}, \tag{10}$$

where $\mathbf{i}_\mathrm{o}$ and $\mathbf{v}_\mathrm{o}$ are vectors of respectively output voltages and currents sensed at the ports; $\mathbf{v}_\mathrm{i}$ and $\mathbf{i}_\mathrm{i}$ are vectors of imposed voltages and currents at these ports; $\mathbf{i}_\mathrm{eq}$ and $\mathbf{v}_\mathrm{eq}$ are vectors of equivalent current and voltage sources of the MPHE; $\mathbf{G}_\mathrm{N}$, $\mathbf{R}_\mathrm{Th}$ are matrices of equivalent conductances and resistances respectively, $\mathbf{K}_\mathrm{I}$ and $\mathbf{K}_\mathrm{V}$ are dimensionless matrices.

D. Combining multiple MPHE Sub-Circuits

Interface currents/voltages at the ports of the MPHEs are computed by connecting the sub-circuits in accordance with the topology of the original circuit. Network equations are composed using the Tableau Approach [21]:

$$\left[\begin{array}{c|c}\mathbf{I} & \mathbf{M}_\mathrm{oi}\\ \hline \mathbf{M}_\mathrm{io} & \mathbf{M}_\mathrm{ii}\end{array}\right]\begin{bmatrix}\mathbf{w}_\mathrm{o}\\ \mathbf{w}_\mathrm{i}\end{bmatrix} = \begin{bmatrix}\mathbf{w}_\mathrm{eq}\\ \mathbf{0}\end{bmatrix}, \tag{11}$$

where $\mathbf{w}_\mathrm{o}$ is the vector of output voltages/currents sensed at the ports of the N MPHEs, $\mathbf{w}_\mathrm{i}$ is the vector of input currents/voltages imposed at these ports, and $\mathbf{w}_\mathrm{eq}$ is the vector of the equivalent voltages/currents sources associated with the MPHEs. $\mathbf{M}_\mathrm{oi}$ is a block matrix composed of the $\mathbf{R}^k_\mathrm{Th}$, $\mathbf{K}^k_\mathrm{V}$, $\mathbf{K}^k_\mathrm{I}$, and $\mathbf{G}^k_\mathrm{N}$ associated with the MPHEs ($1 \leq k \leq N$). $\mathbf{M}_\mathrm{io}$ and $\mathbf{M}_\mathrm{ii}$ are connectivity matrices. $\mathbf{I}$ is the identity matrix.

The generalized post-compensation method [16] states that solving for $\mathbf{w}_\mathrm{i}$ gives, for each sub-circuit, a solution that provides the contribution of the remainder of the network to that sub-circuit. Hence, Eq. 11 offers a simultaneous and exact solution to the whole circuit at the interface ports of the various MPHE sub-circuits. The imposed currents and voltages from the solution $\mathbf{w}_\mathrm{i}$ are then applied to the detailed model of each sub-circuit. Eq. 11 (after appropriate column permutations) can then be reduced using partial Gaussian elimination to form:

$$\mathbf{G}^\mathrm{ind}\mathbf{w}^\mathrm{ind} = \mathbf{g}^\mathrm{ind}, \tag{12}$$

where $\mathbf{w}^\mathrm{ind}$ is a sub-vector with independent variables from $\mathbf{w}_\mathrm{i}$ (redundant terms in $\mathbf{w}_\mathrm{i}$ are omitted). It can be shown that $\mathbf{G}^\mathrm{ind}$ and $\mathbf{g}^\mathrm{ind}$ are respectively composed of terms from $\mathbf{M}_\mathrm{oi}$ and $\mathbf{w}_\mathrm{eq}$ that are summed/subtracted. For certain converter topologies, Eq. 12 is sufficiently small to be solved in real-time on FPGA. Moreover, since its terms are simply sums of pre-computed values, the matrix $\mathbf{G}^\mathrm{ind}$ is easily assembled online. Hence, the simple composition of $\mathbf{G}^\mathrm{ind}$ and its relative small size makes the proposed NTT well tailored to the FPGA context. A short proof of this claim is given in the Appendix.

III. Dealing with Switches in the Model

All switches in the network are modelled using the resistive switch model (R_{on}/R_{off} approach). This model allows the circuit to have a proper tree in any combination of its switch statuses [23], and is known to have a better convergence for naturally commutating switches [24]. It is straightforward to use when dealing with forced commutated switches (IGBTs, for instance), but necessitates an iterative algorithm when confronted with naturally commutating switches, such as diodes.

A. Iterative Solution to Natural Commutation in Switches

Diodes are widely used in power converters to provide rectification and free-wheeling capabilities. In this paper, natural commutation is simulated using the state machine technique illustrated in Fig. 2.a [24], [25]. Hence, updating a diode's status necessitates the sign of the current and voltage at the switch terminals. By using a resistive model of the switch (R_{on}/R_{off} approach), computing the sign of the voltage is

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TIE.2014.2365752, IEEE Transactions on Industrial Electronics

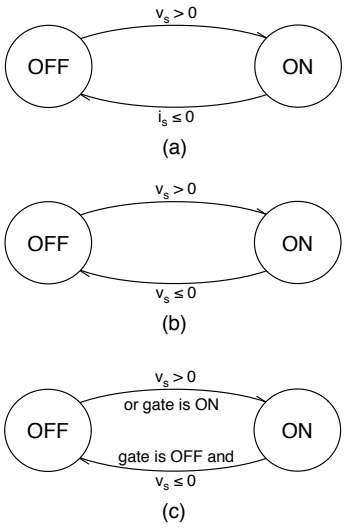

Fig. 2. Diode status updating state machines.

sufficient, since we have $i_{sw} = R_{sw} v_{sw}$, with $R_{sw} > 0$. This leads to the state machine of Fig. 2.b, and adds a new set of equations to Eq. 9:

$$\mathbf{v}_{sw_n} = \mathbf{W}^{\sigma}_{\mathbf{swu}} \mathbf{u}_n + \mathbf{W}^{\sigma}_{\mathbf{swj}} \, \mathbf{j}_n, \tag{13}$$

where $\mathbf{v}_{sw_n}$ is a vector of switch voltages, in which each row is attached to a naturally switching device. The matrices $\mathbf{W}^{\sigma}_{\mathbf{swu}}$ and $\mathbf{W}^{\sigma}_{\mathbf{swj}}$ are expressed accordingly from inline node voltage equations obtained from Eq. 8.

When the diode is connected in parallel to a forced commutation switch such as an IGBT, both devices account for a single switch device and the state machine of Fig. 2.c is used instead, where the gate signal is accounted for.

The switch updating state machine is implemented using an iterative approach, which is done as follows. First, the simulation algorithm reads gate signals ($\mathbf{c}_n$) and sets forced commutated switches in accordance. The algorithm then iterates a given number of times over Eq. 13 to update the status of naturally commutated switches. The number of iterations depends upon the topology of the converter and its operation mode. In offline simulations, the algorithm can iterate until convergence is reached. In the context of a real-time simulation, the computation time should not exceed the time-step of the simulation, hence the number of iterations is forced to a predefined maximal value.

B. NTT Simulation Algorithm

Fig. 3 presents the flowchart for the simulation algorithm using the NTT and the iterative switch updating method. By splitting the circuit into separate parts, this formulation of the algorithm treats subsets of switches independently, improves the computational parallelism and allows complex converters to be simulated in real-time. By computing sub-circuit equations after solving for interface voltages/currents ($\mathbf{w}_n^{\mathrm{ind}}$), the simulation results are as accurate as if network equations were computed for the whole circuit. Hence, the

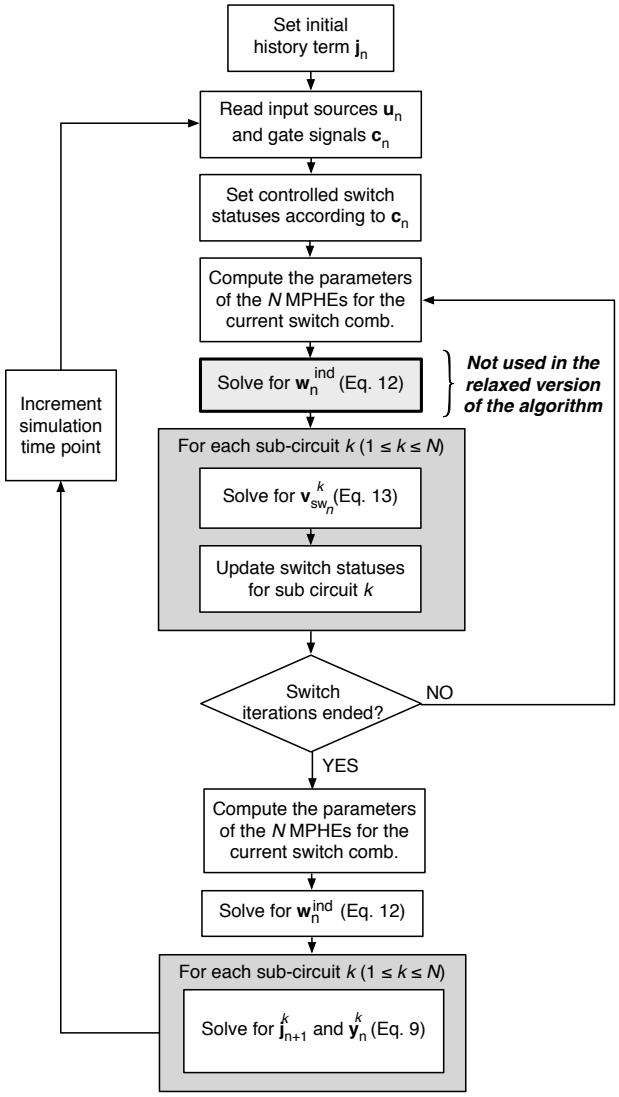

Fig. 3. Flowchart describing the simulation algorithm using the NTT and the iterative switch status updating method.

proposed NTT ensures correct behaviour of the whole power converter model, regardless of its operating mode or topology.

The algorithm of Fig. 3 requires Eq. 12 to be solved at each iteration of the switch updating loop. This constraint can be time-consuming, even if $\mathbf{G}^{\mathrm{ind}}$ is small. In certain situations, this requirement can be relaxed. This is the case when interface voltages/currents are tied to state variables of the circuit. In such circumstances, the algorithm of Fig. 3 is modified by removing the recurring solution to Eq. 12. Instead, $\mathbf{w}_n^{\mathrm{ind}}$ is computed only once, when correct switch statuses are obtained. That value is then used at the following simulation time point during the switch updating loop.

IV. TARGET CIRCUIT

A. Circuit Description

This section presents the target circuit shown in Fig. 4.a: it consists of an NPC converter feeding a three-phrase RLE load. The NPC is a good example of a converter for which the NTT

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TIE.2014.2365752, IEEE Transactions on Industrial Electronics

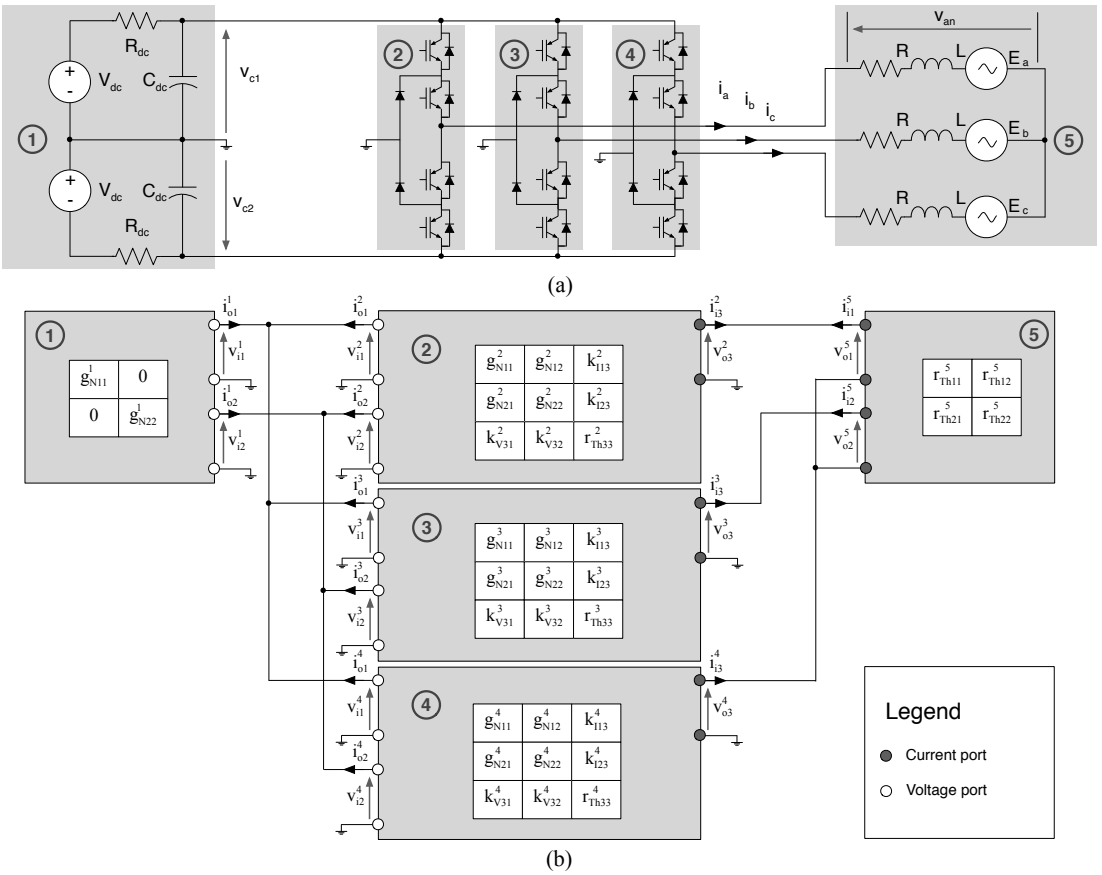

Fig. 4. NPC circuit example (a) original circuit with partitioning into 5 sub-circuits; (b) The 5 MPHEs interconnected with respect to the original topology.

offers good performances on FPGA: the circuit contains 18 switches, thus the memory requirements to store all possible combinations of switch statuses is 2^{18} times Eq. 9 and Eq. 13, which is about 2 Gb of data in single precision arithmetic, whereas an FPGA contains at best tens Mb of embedded memory. By tearing up each arm of the converter, the number of combinations reduces to 2^6 per arm. This yields memory requirement to bearable levels, i.e. less than $10\ \%$ of the available memory resources found on a Virtex 6.

The converter of Fig. 4.a is torn into 5 distinct sub-circuits identified by a circled number and a greyed out background. Each sub-circuit is converter to its MPHE then connected to each other, as shown in Fig. 4.b. The nature of the ports is also shown: dark grey circles stand for current ports and white circles for voltage ports. Out of the 26 port variables, the torn network is governed by the four independent variables: $\mathrm{v}^1_{\mathrm{i}_1}$, $\mathrm{v}^1_{\mathrm{i}_2}$, $\mathrm{i}^2_{\mathrm{i}_3}$, and $\mathrm{i}^3_{\mathrm{i}_3}$. Hence, Eq. 12 yields Eq. 14, where:

$$\mathbf{w}^{\mathrm{ind}} = \left[\mathrm{v}^1_{\mathrm{i}_1}, \mathrm{v}^1_{\mathrm{i}_2}, \mathrm{i}^2_{\mathrm{i}_3}, \mathrm{i}^3_{\mathrm{i}_3}\right]^T. \tag{15}$$

TABLE I
NPC CIRCUIT PARAMETERS

Parameters	Value
SPS Solver	ode23tb
Simulation Sample Time	750 ns
SPWM Carrier Frequency	400 Hz
SPWM Modulation Frequency	30 kHz
AC Voltage Frequency	400 Hz
DC Voltage ($\mathrm{V_{dc}}$)	100 V
$\mathrm{R_{dc}}, \mathrm{R}$	1 Ω
$\mathrm{C_{dc}}$	1 μF
L	1 mH

The terms in $\mathbf{G}^{\mathrm{ind}}$ and $\mathbf{g}^{\mathrm{ind}}$ in Eq. 14 are clearly obtained from additions/subtractions of the MPHE parameters, as claimed. This will always be the case when the mutual independence of input and output currents (and reps. input and output voltages) at the current and voltage ports is guaranteed. It is worth mentioning that the interface currents/voltages identified in Eq. 15 are tied to state variables. This specificity will be used to test the relaxed NTT algorithm.

$$\begin{bmatrix} \mathrm{g}^1_{\mathrm{N}11}+\mathrm{g}^2_{\mathrm{N}11}+\mathrm{g}^3_{\mathrm{N}11}+\mathrm{g}^4_{\mathrm{N}11} & \mathrm{g}^2_{\mathrm{N}12}+\mathrm{g}^3_{\mathrm{N}12}+\mathrm{g}^4_{\mathrm{N}12} & \mathrm{k}^2_{\mathrm{I}13}-\mathrm{k}^4_{\mathrm{I}13} & \mathrm{k}^3_{\mathrm{I}13}-\mathrm{k}^4_{\mathrm{I}13} \\ \mathrm{g}^2_{\mathrm{N}21}+\mathrm{g}^3_{\mathrm{N}21}+\mathrm{g}^4_{\mathrm{N}21} & \mathrm{g}^1_{\mathrm{N}22}+\mathrm{g}^2_{\mathrm{N}22}+\mathrm{g}^3_{\mathrm{N}22}+\mathrm{g}^4_{\mathrm{N}22} & \mathrm{k}^2_{\mathrm{I}23}-\mathrm{k}^4_{\mathrm{I}23} & \mathrm{k}^3_{\mathrm{I}23}-\mathrm{k}^4_{\mathrm{I}23} \\ \mathrm{k}^2_{\mathrm{V}31}-\mathrm{k}^4_{\mathrm{V}31} & \mathrm{k}^2_{\mathrm{V}32}-\mathrm{k}^4_{\mathrm{V}32} & \mathrm{r}^2_{\mathrm{Th}33}+\mathrm{r}^4_{\mathrm{Th}33}+\mathrm{r}^5_{\mathrm{Th}11} & \mathrm{r}^4_{\mathrm{Th}33}+\mathrm{r}^5_{\mathrm{Th}12} \\ \mathrm{k}^3_{\mathrm{V}31}-\mathrm{k}^4_{\mathrm{V}31} & \mathrm{k}^3_{\mathrm{V}32}-\mathrm{k}^4_{\mathrm{V}32} & \mathrm{r}^4_{\mathrm{Th}33}+\mathrm{r}^5_{\mathrm{Th}21} & \mathrm{r}^3_{\mathrm{Th}33}+\mathrm{r}^4_{\mathrm{Th}33}+\mathrm{r}^5_{\mathrm{Th}22} \end{bmatrix} \begin{bmatrix} \mathrm{v}^1_{\mathrm{i}_1} \\ \mathrm{v}^1_{\mathrm{i}_2} \\ \mathrm{i}^2_{\mathrm{i}_3} \\ \mathrm{i}^3_{\mathrm{i}_3} \end{bmatrix} = \begin{bmatrix} \mathrm{i}^1_{\mathrm{eq}_1} \\ \mathrm{i}^1_{\mathrm{eq}_2} \\ -\mathrm{v}^5_{\mathrm{eq}_1} \\ -\mathrm{v}^5_{\mathrm{eq}_2} \end{bmatrix} \tag{14}$$

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TIE.2014.2365752, IEEE Transactions on Industrial Electronics

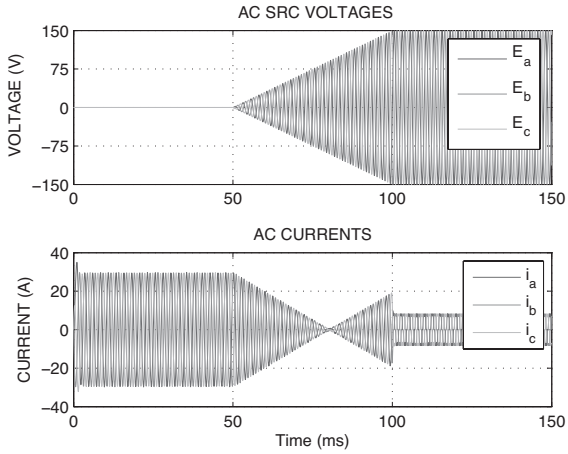

Fig. 5. AC voltage profile and induced current signals from the simulation of the NPC converter SPS model.

TABLE II
AVERAGE RELATIVE ERROR FOR VARIOUS MODELS VS. SPS REFERENCE

Variable	FC1	FC2	FC3	TC2	RTC2
v_{c1}	3.579%	0.185%	0.185%	0.185%	0.185%
v_{c2}	3.578%	0.185%	0.185%	0.185%	0.185%
i_a	15.576%	0.431%	0.430%	0.431%	0.220%
i_b	14.356%	0.398%	0.406%	0.398%	1.058%
i_c	22.386%	0.441%	0.443%	0.441%	1.163%

B. Offline Simulation Results

A three-phase RLE load has been selected for testing purposes. The voltage sources (E_a, E_b, E_c) are useful to create natural rectification conditions. The frequency and amplitude of the AC sources are controlled independently from the circuit conditions. By varying the AC voltage profile as shown in Fig. 5, three operation modes of the NPC converter are induced. During the first phase ($0 \leq t \leq 50$ ms), AC voltages are kept at 0V: the NPC converter acts as an inverter. During the second phase (50 ms $\leq t \leq 100$ ms), AC voltages are increased from 0V to 150V peak. At the beginning of this sequence, the NPC is operating in inverter mode; as AC voltage amplitudes increase, the NPC moves to rectifying mode. During the third phase (100 ms $\leq t \leq 150$ ms), the SPWM is stopped while AC voltages are kept at 150V peak, thus yielding natural rectification mode. Fig. 5 presents the resulting current waveforms at the AC side.

To assess the performance of the NTT, an SPS reference model was designed using the parameters of Table I. Matlab scripts were written to execute the full circuit (FC) model as well as the NTT on torn circuit (TC) models. The FC model is used to evaluate the number of iterations required for the switch updating algorithm. The TC is simulated with and without the relaxed approach. Table II presents the average relative errors observed on circuit variables. FCi models are Full Circuit models (no NTT), where i refers to the number of iterations. Our numerical experiments show that any number of iterations above 2 has results close to FC2. Hence 2 iterations are sufficient to simulate the NPC converter in any of its modes with appropriate accuracy ($\simeq 1$ % of rel. error).

The TC2 and RTC2 models are Torn Circuit models using 2 iterations, with RTC referring to the relaxed version of the NTT algorithm. Table II shows a perfect match between TC2 and FC2. The relaxed NTT algorithm offers very good performances as well. Table II shows that the results remain close to those of the SPS reference. A slight degradation in comparison to TC2 is observed, yet the average relative error levels lie between 0.2% and 1.2%. In conclusion, the reported relative error levels are good, more so when taking into account the fact that real-time simulation is targeted [5].

V. FPGA IMPLEMENTATION

A. Hardware architecture

Unlike other technologies (CPU, GPU), FPGA programming requires a complete design of a so-called Application Specific Processor (ASP) to handle all the computations. This section describes the ASP developed to simulate the NPC converter on FPGA using the relaxed version of the NTT. The architecture of the ASP implemented on the FPGA is presented in Fig. 6.a. It uses five processing units (PUs).

The ASP proceeds as shown in Fig. 6.b: at the beginning of a time point of the simulation, it reads the inputs ($\mathbf{u}_n$ and $\mathbf{c}_n$) and disposes of interface variables ($\mathbf{w}_{n-1}^{\text{ind}}$). The update switch PU uses $\mathbf{w}_{n-1}^{\text{ind}}$ and $\mathbf{c}_n$ to determine switch statuses ($\mathbf{sw}_n$). The statuses are then used to compose $\mathbf{G}_n^{\text{ind}}$. Meanwhile, $\mathbf{g}_n^{\text{ind}}$ is computed using $\mathbf{w}_{n-1}^{\text{ind}}$ and $\mathbf{u}_n$. $\mathbf{G}_n^{\text{ind}}$ and $\mathbf{g}_n^{\text{ind}}$ are fed to the Gauss-Jordan PU that computes interface variables ($\mathbf{w}_n^{\text{ind}}$), then $\mathbf{w}_n^{\text{ind}}$ is used to compute the outputs ($\mathbf{y}_n$), which consists in driving $\mathbf{w}_n^{\text{ind}}$ to the output and summing the inverse of i_a and i_b to obtain i_c. The latter does not influence the minimal time-step of the simulation as the ASP starts over as soon as $\mathbf{w}_n^{\text{ind}}$ is known.

B. Gauss-Jordan Processing units

All the PUs use a parallel dot-product (DP) design approach [8], except for the Gauss-Jordan (GJ) whose implementation is new. The GJ algorithm is known to proceed in two steps. During Step I, the matrix is converted to echelon form with all diagonal entries equal to 1. During Step II, a back substitution process solves sequentially for each unknown. In order to reduce the latency associated with the algorithm, we propose to skip Step II and to run instead multiple copies of the same problem with permuted columns. However, to keep the memory requirements at bearable levels, the matrices are duplicated and have groups of their columns permuted at the first iteration and every time the rank of the reduced matrices are powers of 2 (2, 4, etc.). Fig. 7 illustrates the proposed modification to the GJ algorithm for a 4×4 matrix. The unknowns are noted x_1, x_2, x_3 and x_4, and the system of equations has a unique solution, that is $[1, 2, 3, 4]$.

At the beginning of the algorithm, the matrix is duplicated and the group of columns 1-2 and 3-4 are permuted, then a partial GJ reduction is performed to eliminate one variable on both systems. For instance, the entry at the second row and second column (7) is replaced by $7 - 3 \times 6/2 = -2$, whereas the same position after the permutation of columns 1-2 and 3-4 (9) is replaced by $9 - 5 \times 8/4 = -1$. The resulting two

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TIE.2014.2365752, IEEE Transactions on Industrial Electronics

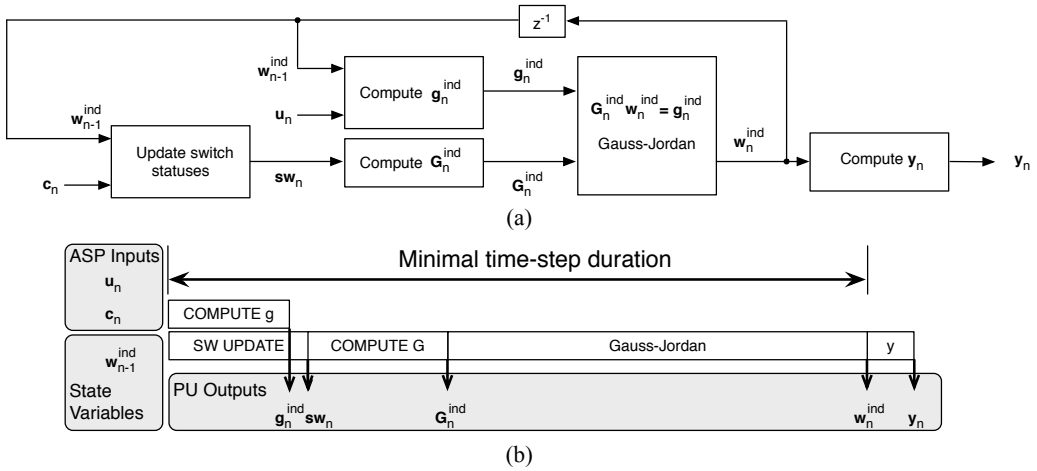

Fig. 6. NTT-based ASP (a) Architecture of the ASP used for the simulation of the NPC circuit; (b) Single time-step computing sequence decomposition.

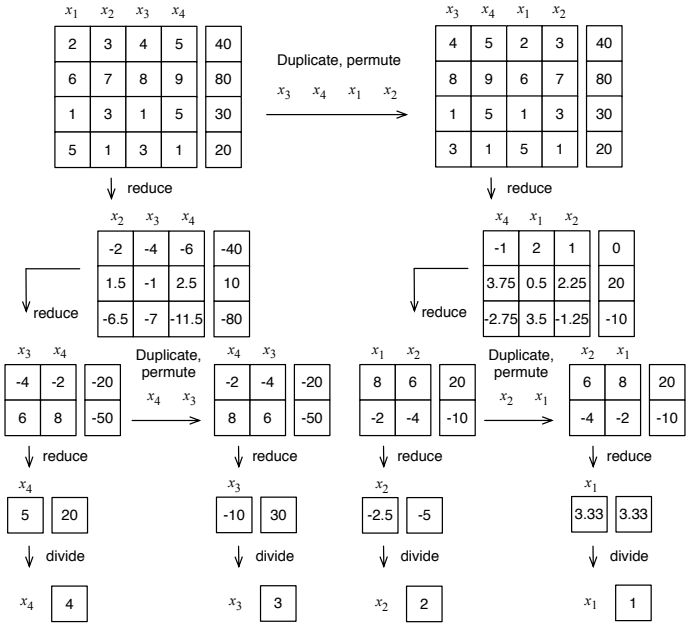

Fig. 7. Illustration of the proposed modification to the GJ algorithm.

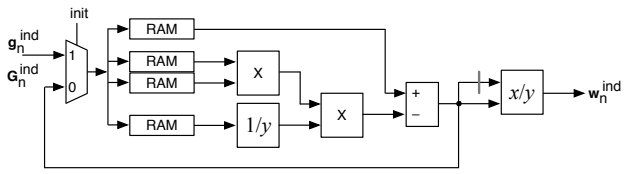

Fig. 8. GJ processing unit.

3×3 matrices are then reduced to 2×2 matrices using partial GJ elimination. The new matrix size is a power of two, so duplications and permutations are performed. After reduction, we end up with four singletons: simple divisions suffice to obtain the aforementioned solution.

Fig. 8 presents the architecture of the PU that executes such a variation of the GJ algorithm. The arithmetic operators are pipelined and can accept new operands while processing older ones, hence multiple copies of a problem can be executed simultaneously. This property allows the reduction of the total latency associated with the execution of the GJ algorithm.

TABLE III
FPGA RESOURCE UTILIZATION, TARGETING THE VIRTEX 6 XC5VLX240T-1 [26] OF THE ML605 FPGA BOARD.

Module	Slices	BRAM	DSP	Min. period	Latency
Compute $\mathbf{g}^{\mathrm{ind}}$	1,671	0	24	2.469 ns	38
Switch update	2,687	20	60	2.466 ns	61
Compute $\mathbf{G}^{\mathrm{ind}}$	987	8	0	2.468 ns	46
Gauss Jordan	1,332	13	32	2.453 ns	182
Compute $\mathbf{y}$	781	0	0	2.460 ns	22
NPC ASP	7,277	41	116	2.488 ns	289
Available	37,680	416	768	N/A	N/A

C. Implementation results

All numerical values in the ASP are represented in floating-point to allow a large dynamic range. The floating-point format used is a non-standard one with intermediate precision (52-bit signed mantissa and 8-bit exponent), while all the additions are carried out using an internal floating-point format called the Self-Alignment Format (SAF) that uses an 80-bit extended mantissa. This methodology has been introduced in [8] whereas the SAF is thoroughly discussed in [27]. Each multiplication consumes 6 DSP blocks and necessitates 5 clock cycles. The reciprocal and the division operators are designed using the Newton-Raphson algorithm [28].

Table III presents the implementation results for the PUs and the ASP when the Virtex 6 LX240T-1 is targeted [26]. The design is performed using the System Generator Matlab/Simulink toolbox from Xilinx. The PUs were designed with a 400 MHz target frequency (2.5 ns minimal clock period), which is quite high in regard to the industrial standards and the literature known to the authors. As shows Table III, the 2.5 ns constraint is met by all the PUs, as well as by the ASP. The ASP consumes about 19% of the reconfigurable logic (slices). We also observe that the memory consumption is around 9% to 10% of the available BRAMs, which meets

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TIE.2014.2365752, IEEE Transactions on Industrial Electronics

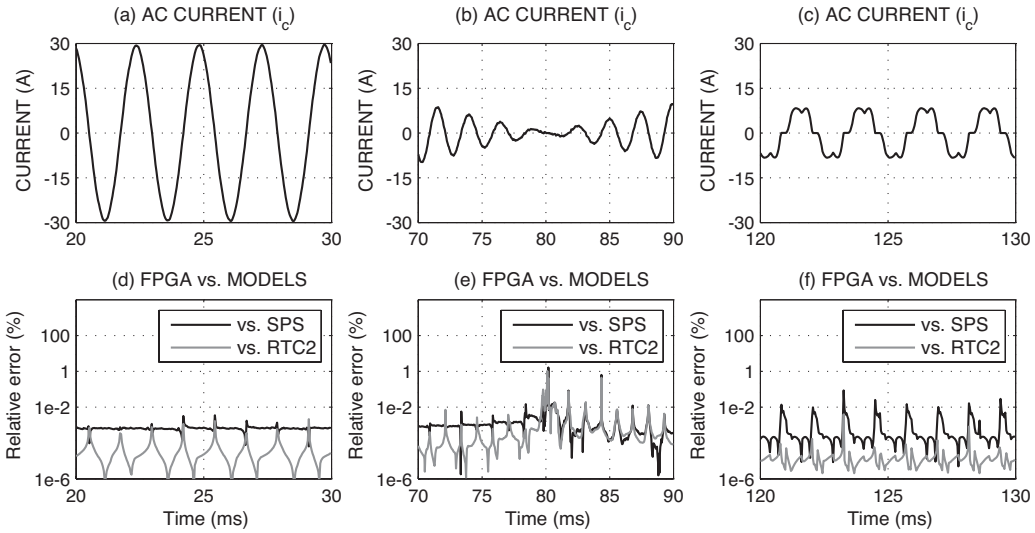

Fig. 9. Zoomed-in captures from the $\mathrm{i_c}$ current, along with the relative errors produced by the ASP with respect to the SPS and RTC2 references.

our expectations. Memory resources are located at the Switch Update PU, the $\mathbf{G}_n^{\mathrm{ind}}$ computing PU and the GJ PU. Regarding the first two PUs, their BRAM consumption is explained by their switch status dependant characteristic. DSP block consumption is about 15% of the available resources, most of which are located at the switch update PU. The $\mathbf{G}_n^{\mathrm{ind}}$ and $\mathbf{y}_n$ computing PUs do not use any DSP block at all, which is explained by the fact that those PUs do not perform any multiplication.

Table III presents the latency of each PU as well as the total latency of the ASP, which is 289 clock cycles. The latency of the ASP comprises those of the Switch Update, the $\mathbf{G}_n^{\mathrm{ind}}$ computing and the GJ PUs. At a clock frequency of 400 MHz, 289 clock cycles yield 722.5 ns, but the time step of the ASP was rounded up to 750 ns. The Gauss-Jordan PU is the most time-consuming unit. This observation is consistent with the algorithm's complexity which grows as $O(N^3)$ with the rank (N) of the matrix. Without our modified version of the GJ algorithm, latency would have doubled and simulation time-step would have been higher than 1 μs.

D. FPGA-based simulation results

The ASP was implemented on an ML-605 board from Avnet. The design was hooked up to a host computer via an ethernet TCP/IP link. The results were then compared using MATLAB to the SPS reference, the FC2 model and to the RTC2 model. Table IV presents the average relative errors resulting from the ASP computations. Fig. 9 presents zoomed-in captures from the $\mathrm{i_c}$ current waveform for each mode of the simulation, along with the evolution of the relative errors with respect to the SPS and RTC2 references. Theses captures confirm the results from Table IV and show that, for the SPS and FC2 references, the maximum relative error is observed when current is close to zero. This observation is explained by the properties of the BE integration rule that is known to exhibit a low accuracy due to its low order. However, the errors remain at acceptable levels ($\simeq 1\%$).

TABLE IV
AVERAGE RELATIVE ERROR FOR FPGA MODEL

Variable	vs. SPS	vs. FC2	vs. RTC2
$\mathrm{v_{c1}}$	0.185%	0.331e-3%	0.277e-3%
$\mathrm{v_{c2}}$	0.185%	0.310e-3%	0.277e-3%
$\mathrm{i_a}$	0.206%	0.177%	0.047%
$\mathrm{i_b}$	1.061%	0.335%	0.023%
$\mathrm{i_c}$	1.142%	0.466%	0.093%

When the FPGA-based simulation is compared to the RTC2 model, we are assessing the computational accuracy of the ASP, which lies between $10^{-6}\%$ and $10^{-2}\%$. Such levels of relative errors are explained by the fact that the implemented floating-point operators are not fully IEEE double precision compliant. However, this design tradeoff comes with area savings and offers very low latency computations.

In conclusion, the results from Table IV and Fig. 9 clearly demonstrate the effectiveness of the NTT and that of the proposed hardware design methodology.

E. Voltage-Switching Behaviour

Certain switch modelling techniques, such as the fixed admittance switch model [11], [12], are known to introduce false transients during voltage switchings. Depending on converter load and switching frequency, those transients do not necessarily affect the overall shape of the current. In order to make sure that such phenomena are not observed in our model, the voltage from the converter to the neutral point of the load ($\mathrm{v_{an}}$, see Fig. 4.a for correspondance) is shown in Fig. 10, along with results from the SPS reference and the absolute errors produced.

The capture of Fig. 10.a is taken from the moment of the simulation where the errors on the current waveform were the most problematic, which occurred around 80 ms of simulation time. The superimposition of the model and the SPS reference shows a perfect match between them, which is confirmed by the plot of Fig. 10.d.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TIE.2014.2365752, IEEE Transactions on Industrial Electronics

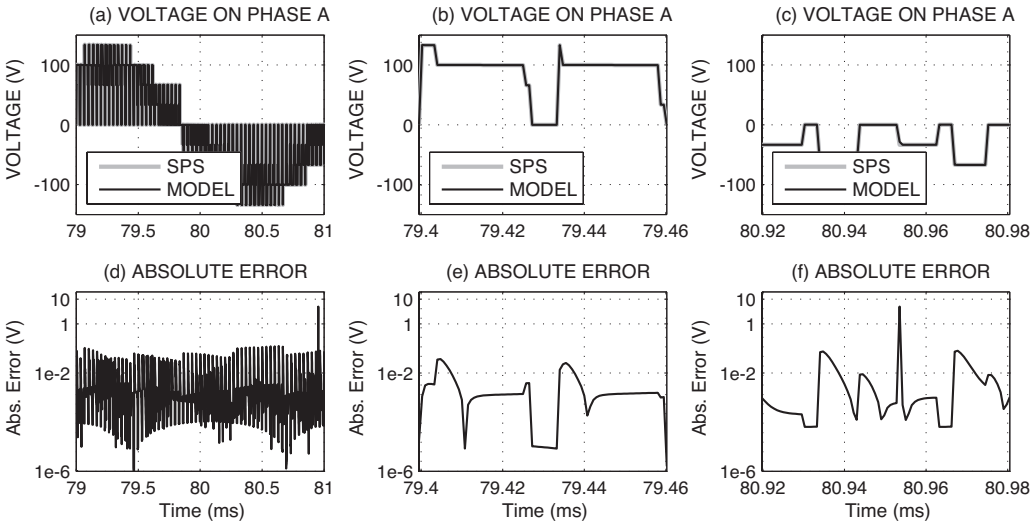

Fig. 10. Voltage from the converter to the neutral point of the load on phase a, superimposed on the result from the SPS reference.

Fig. 10.b shows a close-up of the voltage profile at the beginning of the sequence shown in Fig. 10.a. The perfect match between our model and the SPS reference appears clearly, more so when considering the absolute error associated with this capture, as shown in Fig. 10.e. We also observe that the error reaches its maxima during voltage switchings, but the error remains very acceptable and the model rapidly converges to the desired value, with absolute errors below 0.01V.

Fig. 10.c shows another close-up of the voltage profile — which occurs at the end of the sequence shown in Fig. 10.a this time, where a peak value of the error was observed. The zoomed view confirms that the difference occurrence at a voltage switching point and only lasts for on time-step, as the model converges rapidly toward to the correct value.

VI. Conclusion

As the HIL prototyping approach gains importance for the industry, FPGA-based real-time simulation becomes a key component that helps meeting the timing constraint imposed by high switching frequency power converters. This article presented an original solution to that issue whose formulation results from the development of a Network Tearing Technique, specially tailored to the FPGA context. A very high operating clock frequency is reported for the proposed ASP design, at an acceptable FPGA resource consumption. The article also presented a detailed design methodology that ensures high computational accuracy, high performance, and moderate reconfigurable footprint. It allows < 1 μs time-steps to be reached for the real-time simulation of power converters.

Appendix
Proof for the Claim Regarding the Reduced Network Equations

In Eq. 11, $\mathbf{M}_{\mathrm{io}}$ and $\mathbf{M}_{\mathrm{ii}}$ are connectivity matrices composed of 0s and ± 1 entries. If current ports are connected to each other, and if so are the voltage ports (mutual exclusiveness of input and output variables criterion), then we have:

$$\left[\begin{array}{c|c} \mathbf{M}_{\mathrm{io}} & \mathbf{M}_{\mathrm{ii}} \end{array}\right] = \left[\begin{array}{c|c} \mathbf{0} & \mathbf{M}_{\mathrm{ii}}^{(1)} \\ \hline \mathbf{M}_{\mathrm{io}}^{(2)} & \mathbf{0} \end{array}\right], \tag{16}$$

Moreover, since a set of independent variables $\mathbf{w}^{\mathrm{ind}}$ is present in $\mathbf{w}_{\mathrm{i}}$, and supposing that those are placed at the bottom end of $\mathbf{w}_{\mathrm{i}}$ (which can necessitate the permutations of certain columns of $\mathbf{M}$, as we did in the example of Section II-A), then $\mathbf{M}_{\mathrm{ii}}^{(1)}$ can be built in a way that we guarantees the following:

$$\mathbf{M}_{\mathrm{ii}}^{(1)} = \left[\begin{array}{c|c} \mathbf{I} & \mathbf{M}_{\mathrm{ii}}^{(12)} \end{array}\right]. \tag{17}$$

where $\mathbf{I}$ is a square identity matrix with a number of columns equal to the number of dependent variables in $\mathbf{w}_{\mathrm{i}}$ (hence, $\mathbf{M}_{\mathrm{io}}^{(2)}$ has a number of rows equal to the size of $\mathbf{w}^{\mathrm{ind}}$).

From the above, it results that a partial Gauss-Jordan reduction of $\mathbf{M}$ to its last rows (those associated with $\mathbf{w}^{\mathrm{ind}}$) only affects those rows and leaves the remaining part of the matrix unchanged, thus yielding a matrix $\mathbf{G}^{\mathrm{ind}}$ composed of sums/subtractions of terms from $\mathbf{M}_{\mathrm{oi}}$, as $\mathbf{M}_{\mathrm{io}}^{(2)}$ and $\mathbf{M}_{\mathrm{ii}}^{(12)}$ contain nothing but 0s and ± 1 entries.

References

[1] J. Mahseredjian, V. Dinavahi, and J. Martinez, "Simulation tools for electromagnetic transients in power systems: Overview and challenges," *IEEE Trans. Power Del.*, vol. 24, no. 3, pp. 1657–1669, July 2009.

[2] Y. Levron, H. Kim, and R. Erickson, "Design of EMI filters having low harmonic distortion in high-power-factor converters," *IEEE Trans. Power Electron.*, vol. 29, no. 7, pp. 3403–3413, July 2014.

[3] B. Zhang, K. Zhou, and D. Wang, "Multirate repetitive control for PWM DC/AC converters," *IEEE Trans. Ind. Electron.*, vol. 61, no. 6, pp. 2883–2890, June 2014.

[4] A. Kuperman, U. Levy, J. Goren, A. Zafransky, and A. Savernin, "Battery charger for electric vehicle traction battery switch station," *IEEE Trans. Ind. Electron.*, vol. 60, no. 12, pp. 5391–5399, Dec 2013.

[5] H. F.-Blanchette, T. Ould-Bachir, and J.-P. David, "A state-space modeling approach for the FPGA-based real-time simulation of high switching frequency power converters," *IEEE Trans. Ind. Electron.*, vol. 59, no. 12, pp. 4555–4567, December 2012.

[6] M. Matar and R. Iravani, "FPGA implementation of the power electronic converter model for real-time simulation of electromagnetic transients," *IEEE Trans. Power Del.*, vol. 25, no. 2, pp. 852–860, April 2010.
[7] D. Majstorovic, I. Celanovic, N. D. Teslic, N. Celanovic, and V. A. Katic, "Ultralow-latency hardware-in-the-loop platform for rapid validation of power electronics designs," *IEEE Trans. Ind. Electron.*, vol. 58, no. 10, pp. 4708–4716, Oct. 2011.
[8] T. Ould-Bachir, C. Dufour, J. Bélanger, J. Mahseredjian, and J.-P. David, "A fully automated reconfigurable calculation engine dedicated to the real-time simulation of high switching frequency power electronic circuits," *Math. Comput. Simulat.*, vol. 91, pp. 167–177, 2013.
[9] H. Jin, "Behavior-mode simulation of power electronic circuits," *IEEE Trans. Power Electron.*, vol. 12, no. 3, pp. 443–452, May 1997.
[10] Y. Inaba, S. Cense, T. Ould-Bachir, H. Yamashita, and C. Dufour, "A dual high-speed PMSM motor drive emulator with finite element analysis on FPGA chip with full fault testing capability," in *European Conf. Power Electron. App.*, Nottingham, UK, Sept. 2011, pp. 1–10.
[11] S. Hui and C. Christopoulos, "A discrete approach to the modeling of power electronic switching networks," *IEEE Trans. Power Electron.*, vol. 5, no. 4, pp. 398–403, Oct. 1990.
[12] P. Pejovic and D. Maksimovic, "A method for fast time-domain simulation of networks with switches," *IEEE Trans. Power Electron.*, vol. 9, no. 4, pp. 449–456, July 1994.
[13] P. Le-Huy, S. Guerette, L. A. Dessaint, and H. Le-Huy, "Real-time simulation of power electronics in power systems using an FPGA," in *Canadian Conf. Electr. Comput. Eng.*, May 2006, pp. 873–877.
[14] M. Dagbagi, L. Idkhajine, E. Monmasson, and I. Slama-Belkhodja, "FPGA implementation of power electronic converter real-time model," in *Int. Symp. Power Electro. Electr. Drives, Automation Motion*, June 2012, pp. 658–663.
[15] L. Chua and L.-K. Chen, "Diakoptic and generalized hybrid analysis," *IEEE Trans. Circuits Syst.*, vol. 23, no. 12, pp. 694–705, 1976.
[16] J. Mahseredjian, S. Lefebvre, and D. Mukhedkar, "Power converter simulation module connected to the EMTP," *IEEE Trans. Power Syst.*, vol. 6, no. 2, pp. 501–510, 1991.
[17] J. Marti, L. Linares, J. Calvino, H. Dommel, and J. Lin, "OVNI: an object approach to real-time power system simulators," in *Proc. Int. Conf. Power Syst. Technol.*, Aug. 1998, pp. 977–981.
[18] K. Strunz and E. Carlson, "Nested fast and simultaneous solution for time-domain simulation of integrative power-electric and electronic systems," *IEEE Trans. Power Del.*, vol. 22, no. 1, pp. 277–287, 2007.
[19] C. Dufour, J. Mahseredjian, and J. Bélanger, "A combined state-space nodal method for the simulation of power system transients," *IEEE Trans. Power Del.*, vol. 26, no. 2, pp. 928–935, April 2011.
[20] J. Mahseredjian, S. Dennetière, L. Dubé, B. Khodabakhchian, and L. Gérin-Lajoie, "On a new approach for the simulation of transients in power systems," *Electric Power Systems Research*, vol. 77, no. 11, pp. 1514–1520, 2007, selected Topics in Power System Transients - Part II.
[21] G. Hachtel, R. Brayton, and F. Gustavson, "The sparse tableau approach to network analysis and design," *IEEE Transactions on Circuit Theory*, vol. 18, no. 1, pp. 101 – 113, Jan. 1971.
[22] E. Hairer and G. Wanner, *Solving ordinary differential equations II: Stiff and differential-algebraic problems*, ser. Solving ordinary differential equations. Springer, 2010.
[23] L. Chua and P. Lin, *Computer-aided analysis of electronic circuits: algorithms and computational techniques*, ser. Prentice-Hall series in electrical and computer engineering. Prentice-Hall, 1975.
[24] F. E. Cellier and E. Kofman, *Continuous System Simulation*. Secaucus, NJ, USA: Springer-Verlag New York, Inc., 2006.
[25] J. Hautier and J. Caron, *Convertisseurs statiques: méthodologie causale de modélisation et de commande*, ser. Méthodes et pratiques de l'ingénieur. Technip, 1999.
[26] Xilinx, *DS150 v2.3: Virtex-6 Family overview*, March 2011.
[27] T. Ould-Bachir and J.-P. David, "Self-alignment schemes for the implemention of addition-related floating-point operators," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 6, no. 1, pp. 1–21, 2013.
[28] M. Flynn, "On division by functional iteration," *IEEE Trans. Comp.*, vol. C-19, no. 8, pp. 702–706, Aug. 1970.

Tarek Ould-Bachir (S'05–M'08) received the B.Eng. M.A.Sc. and Ph.D. degrees in electrical engineering from École Polytechnique de Montréal, Montreal, Qc, Canada, in 2005, 2008 and 2013 respectively. From 2013 to 2014, he was a post-doctoral fellow at École de Technologie Supérieure, Montreal, where his research focused on high performance FPGA-based computing engines for the real-time simulation of power converters. Since 2005, he has been lecturing undergraduate courses at École Polytechnique de Montréal. From 2007 to 2009, he worked as an FPGA Application Specialist with Opal-RT Technologies. Since september 2014, he is a Senior Simulation Specialist with the same company. His research interests include real-time simulation, power electronics, FPGAs, and digital arithmetic.

Handy Fortin Blanchette (S'07-M'10) received the B.Eng., M.Eng. and Ph.D. degrees in electrical engineering from the École de Technologie Supérieure (ETS), Montreal, Qc, Canada, in 2001, 2003 and 2010, respectively. From 1998 to 2000, he was with the Bombardier Transport-ETS Research Laboratory, where he worked on high-power traction systems. From 2001 to 2003, he was involved in the development of an electrical drive library for Simulink. From 2007 to 2010, he was with OPAL-RT Technologies, where he led power electronics real-time simulation projects. From 2010 to 2011, he was a Visiting Scholar with the Center for Power Electronics Systems, Virginia Polytechnic Institute and State University, Blacksburg, VA, USA, where he was involved in the packaging of converters for aircraft applications. He is currently an Associate Professor of electrical engineering with ETS. His current research interests include EMI, FPGA-based real-time simulations, and high-density power converter packaging.

Kamal Al-Haddad (S'82-M'88-SM'92-F'07) received the B.Sc.A. and M.Sc.A. degrees from the University of Québec Trois-Rivières, Canada, in 1982 and 1984, respectively, and the Ph.D. degree from the Institut National Polytechnique, Toulouse, France, in 1988. Since June 1990, he has been a Professor with the Electrical Engineering Department, École de Technologie Supérieure (ETS), Montreal, QC, where he has been the holder of the Canada Research Chair in Electric Energy Conversion and Power Electronics since 2002. He is a Consultant and has established very solid link with many Canadian industries. His fields of interest are in high efficient power converters, harmonics and reactive power control, switch mode and resonant converters including the modelling, control, and development of prototypes for industrial applications. Prof. Al-Haddad is a fellow member of the Canadian Academy of Engineering. He is IEEE IES President Elect, Associate editor of the Transactions on Industrial Informatics and IES Distinguished Lecturer.